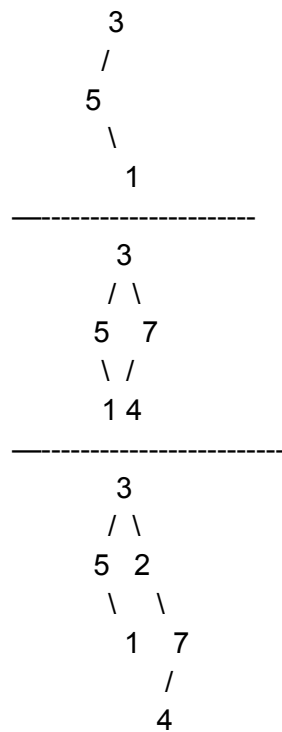


What gets printed by the first part of the code is 3 trees separated by dashed lines
It looks like this:



```
from BstNode import BstNode
```

```
class BST(BstNode):
    def __init__(self, key):
        super().__init__(key)

    def find(self, value):
        if value == self.key:
            return True
        elif value < self.key:
            if self.left is None:
                return False
            return self.left.find(value)
        else:
            if self.right is None:
                return False
            return self.right.find(value)

    def insert(self, value):
        if value < self.key:
            if self.left is None:
                self.left = BST(value)
```

```

        else:
            self.left.insert(value)
    elif value > self.key:
        if self.right is None:
            self.right = BST(value)
        else:
            self.right.insert(value)
    # if value == self.key, do nothing (no duplicates)

def _min_value_node(self):
    current = self
    while current.left is not None:
        current = current.left
    return current

def remove(self, value):
    # special case: removing the root itself
    if value == self.key:
        # case 1: leaf
        if self.left is None and self.right is None:
            return None

        # case 2: only right child
        if self.left is None:
            return self.right

        # case 3: only left child
        if self.right is None:
            return self.left

        # case 4: two children
        successor = self.right._min_value_node()
        self.key = successor.key
        self.right = self.right.remove(successor.key)
        return self

    elif value < self.key:
        if self.left is not None:
            self.left = self.left.remove(value)
        return self

    else:
        if self.right is not None:
            self.right = self.right.remove(value)

```

return self